Envío de Evidencias MVP: PichangaBook Arequipa

Avance de Backend, Frontend, Diagramas y Planificación

Formación de Empresas de Base Tecnológica I — Universidad Católica San Pablo

Rodrigo Sierra Samuel Iman

5 de noviembre de 2026

1 Resumen del Avance del MVP

En este informe presentamos las evidencias actuales del Producto Mínimo Viable (MVP) de **PichangaBook Arequipa** (repositorio `github.com/RodrigoSierr/PichangaBook`). El monorepo integra tres carpetas: `/backend-nestjs` (API REST y WebSockets en NestJS con PostgreSQL), `/web-admin-tailwind` (panel web para dueños de complejos) y `/mobile-flutter` (pantallas en Flutter/Dart junto al cliente móvil interactivo de demostración). En esta entrega se encuentran implementados el esquema relacional de 13 tablas, la prevención de reservas cruzadas con índices GiST, la sincronización en tiempo real por Socket.io, el cálculo de distancias con Haversine y el chat interno con redacción de teléfonos; mientras que los cobros de pasarela (Culqi/Yape/Plin), el código OTP por SMS, el número proxy y la firma de subida de videos operan actualmente mediante adaptadores de demostración locales cuya integración con servicios externos reales está programada para la siguiente semana.

2 Diagramas del Sistema

2.1 Diagrama de Arquitectura Backend–Frontend

El diagrama muestra los módulos configurados en `src/app.module.ts`, diferenciando la persistencia y eventos en tiempo real de los adaptadores que hoy corren en modo demostración:

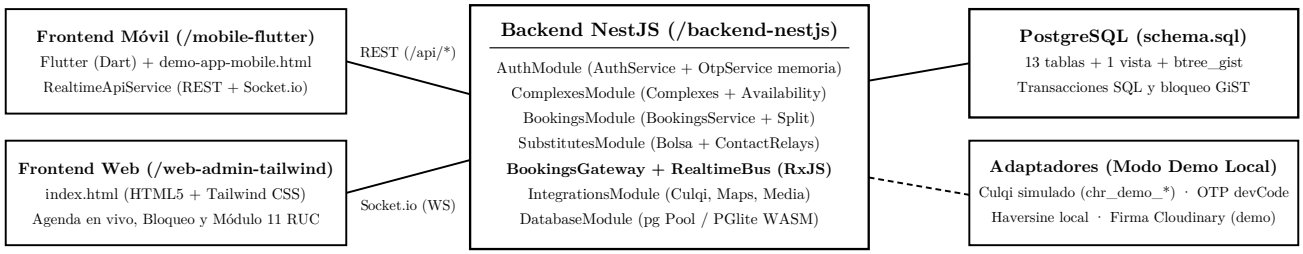


Figura 1: Diagrama de arquitectura del monorepo diferenciando los módulos implementados y los adaptadores en modo demo.

2.2 Diagrama Entidad-Relación de la Base de Datos (PostgreSQL)

El diagrama Entidad-Relación representa las **13 tablas independientes** (más la vista `v_complex_ratings`) definidas en `backend-nestjs/schema.sql` con sus llaves primarias (`PK`), foráneas (`FK`) y restricciones (`UQ`):

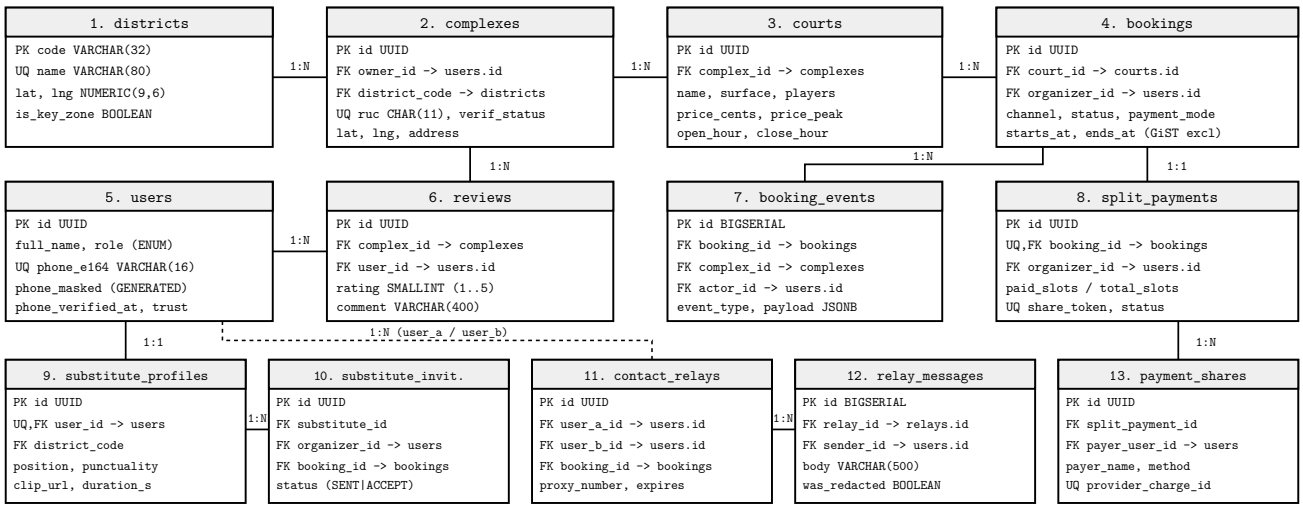


Figura 2: Diagrama Entidad-Relación con las 13 tablas independientes definidas en `backend-nestjs/schema.sql`.

3 Evidencias de Código (Backend, Base de Datos y Frontend)

3.1 Esquema en PostgreSQL y restricción anti-doble reserva (`schema.sql`)

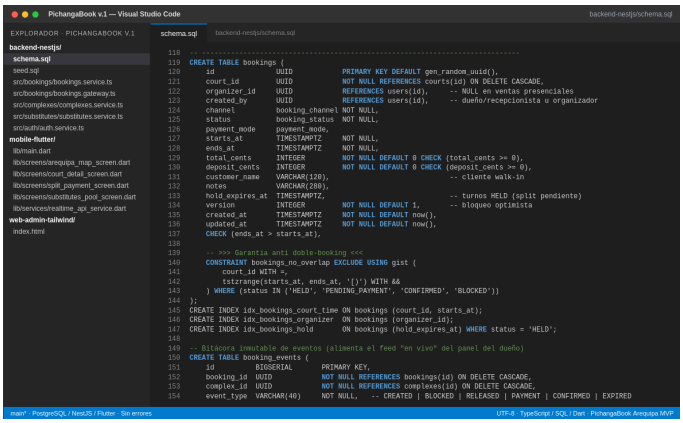


Figura 3: Código SQL de la tabla `bookings` y su restricción GiST en `backend-nestjs/schema.sql`.

En la captura se observa el archivo `backend-nestjs/schema.sql` (líneas 118 a 154) con la definición de las tablas `bookings` y `booking_events` dentro del esquema de 13 tablas.

En las líneas 140 a 143 programamos `CONSTRAINT bookings_no_overlap EXCLUDE USING gist` con la extensión `btree_gist`, impidiendo en PostgreSQL que dos turnos activos (`HELD`, `PENDING_PAYMENT`, `CONFIRMED` o `BLOCKED`) se solapen en la misma cancha. Aunque al crear un turno `HELD` se calcula `hold_expires_at` (+30 min), la tarea programada (Cron Job) para liberar automáticamente reservas vencidas está pendiente para la próxima semana.

3.2 Lógica de reservas y pagos divididos en modo demostración (`bookings.service.ts`)

El método `createBooking` de `BookingsService` (en `src/bookings/bookings.service.ts`) ejecuta la inserción de la reserva, del registro `split_payments` y de la bitácora `booking_events` dentro de una transacción SQL (`this.db.transaction(...)`).

Como muestra la captura (líneas 91 a 128), al iniciar un cobro `SPLIT` se invoca `this.culqi.createCharge(...)` para el primer cupo y se inserta en `payment_shares`. Actualmente este cobro funciona en **modo demostración**: sin `CULQI_SECRET_KEY`, `CulqiService` devuelve un cargo simulado (`chr_demo_*`) sin requerir aún un `sourceId` real tokenizado en el cliente.

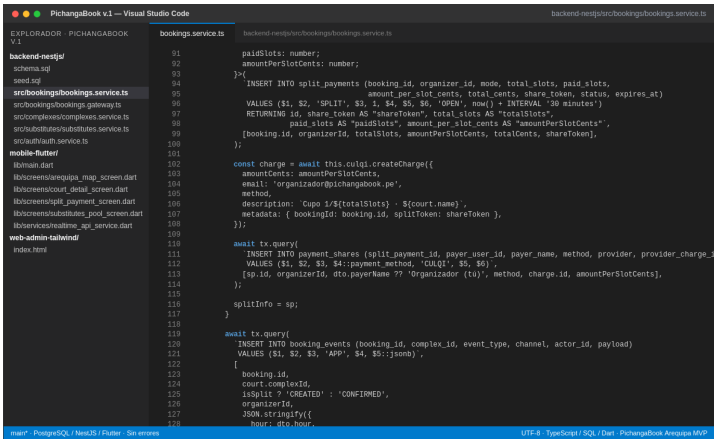


Figura 4: Transacción de reserva y creación de pago dividido (modo demo) en `bookings.service.ts`.

3.3 Gateway de sincronización en tiempo real con Socket.io (`bookings.gateway.ts`)

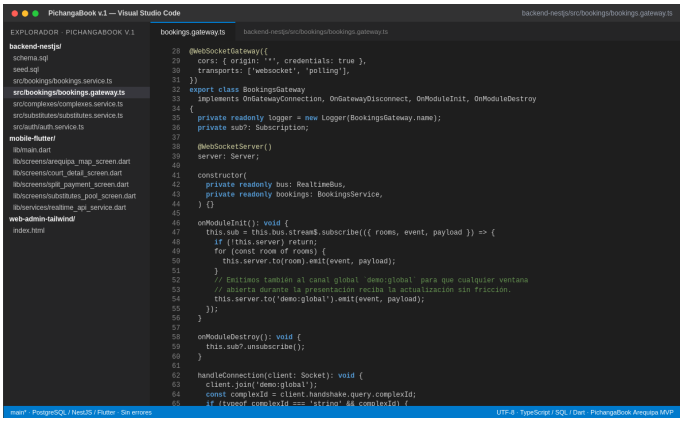


Figura 5: Implementación de `BookingsGateway` con Socket.io en NestJS.

Para que los bloqueos hechos en recepción y los abonos del pago dividido se reflejen al instante sin recargar la pantalla, implementamos la clase `BookingsGateway` en `src/bookings/bookings.gateway.ts` con `@nestjs/websockets` y **Socket.io**.

En las líneas 46 a 65 se observa cómo `onModuleInit()` se suscribe al bus reactivo `RealtimeBus` (RxJS) y difunde los eventos `slot:updated`, `split:updated` y `feed:event` hacia la sala del complejo (`complex:<id>`) y al canal `demo:global`.

3.4 Código del Frontend móvil en Flutter (`arequipa_map_screen.dart`)

En la carpeta `/mobile-flutter/lib` implementamos las pantallas móviles en **Dart / Flutter** (`arequipa_map_screen.dart`, `court_detail_screen.dart`, `split_payment_screen.dart` y `substitutes_pool_screen.dart`) conectadas al cliente `RealtimeApiService`, además del espejo interactivo web (`demo-app-mobile.html`) usado para las pruebas en navegador.

La captura muestra las primeras 35 líneas de `arequipa_map_screen.dart`, donde se define el `StatefulWidget` con `google_maps_flutter`, las coordenadas de Arequipa (`LatLng(-16.3880, -71.5430)`) y los filtros por distrito.

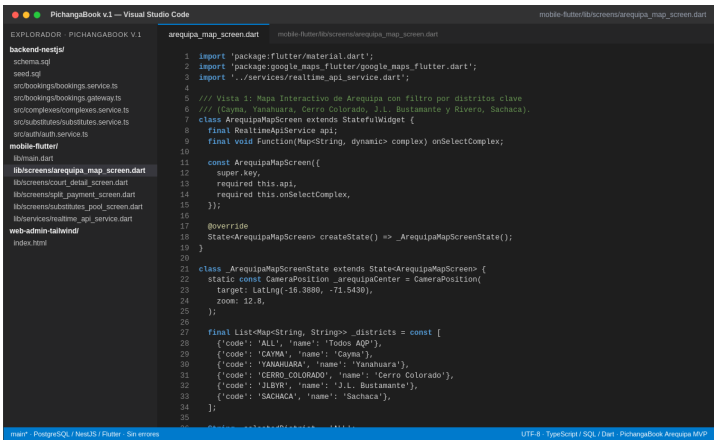


Figura 6: Código en Flutter (Dart) de la pantalla del mapa de Arequipa.

4 Evidencias Visuales del Frontend Conectado al Backend

4.1 Panel web para dueños de complejos deportivos y validación de formato RUC

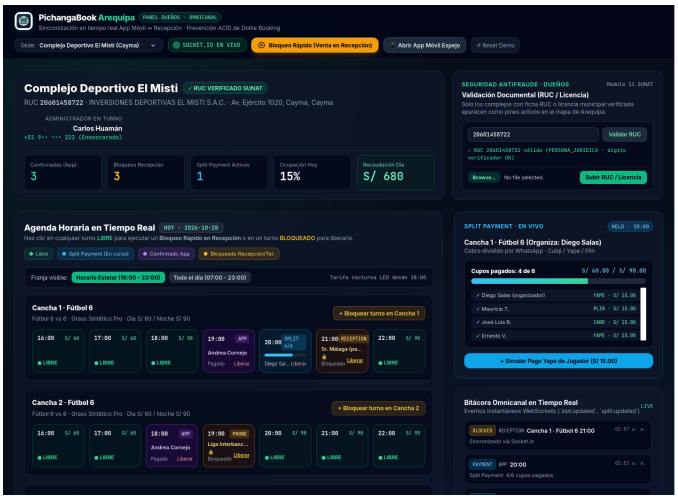


Figura 7: Panel web de administración para dueños de complejos corriendo en el navegador.

El panel web (`/web-admin-tailwind/index.html`), maquetado con **HTML5** y **Tailwind CSS**, consume `GET /api/complexes` y `GET /api/owner/complexes/:id/dashboard` para mostrar la ocupación del día, la parrilla horaria por cancha y la bitácora de eventos en vivo.

Incluye el módulo de verificación documental: `GET /api/ruc/:ruc/validate` comprueba localmente el formato de 11 dígitos y el dígito verificador mediante el **algoritmo módulo 11** (sin consultar en línea a SUNAT), y `POST /api/complexes/:id/verification-document` guarda el archivo en disco local (con autoaprobación tras unos segundos cuando `demoMode` está activo).

4.2 Bloqueo rápido de turnos en recepción

Cuando ingresa un alquiler presencial en boletería o por llamada telefónica, el encargado abre este modal en el panel web, selecciona la cancha, la hora, el canal (`RECEPTION` o `PHONE`) y el nombre del cliente.

Al confirmar, se envía `POST /api/bookings/reception-block` hacia `BookingsController`, insertando el turno con estado `BLOCKED` en `bookings` y emitiendo `slot:updated` por Socket.io. Para agilizar la demostración local, esta ruta y el dashboard del dueño tienen temporalmente el decorador `@Public()` (usando `DEMO_OWNER_ID` por defecto), quedando pendiente exigir el guard de rol `OWNER`.

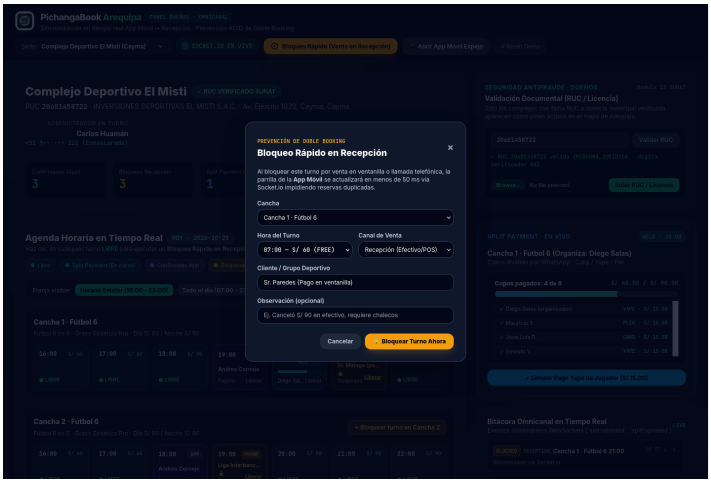


Figura 8: Modal de bloqueo rápido en recepción para ventas presenciales o telefónicas.

4.3 Mapa de complejos y filtros por distritos de Arequipa



Figura 9: Vista móvil del mapa de Arequipa y complejos filtrados por distrito.

La vista inicial muestra el mapa interactivo de Arequipa con pines que indican la tarifa mínima por hora de cada complejo deportivo y chips superiores para filtrar por distrito, alimentados desde la tabla `districts` mediante `GET /api/districts`.

Al elegir un distrito, la interfaz invoca `GET /api/complexes?district=...` en `ComplexesController`, que filtra en PostgreSQL los complejos con `verification_status = 'VERIFIED'`, obtiene su promedio de reseñas desde la vista `v_complex_ratings` y ordena los resultados calculando la distancia en kilómetros con la fórmula de Haversine (`MapsService.haversineKm`).

4.4 Detalle de cancha y parrilla de disponibilidad en vivo

Al seleccionar un complejo se consulta `GET /api/complexes/:id` y `GET /api/courts/:id/availability` (servido por `AvailabilityService.courtGrid`), mostrando las canchas activas del local (Fútbol 6 y Fútbol 8) y las opiniones almacenadas en la tabla `reviews`.

La parrilla horaria diferencia los cuatro estados devueltos por el backend: disponible en verde (`FREE`), reservado por app en morado (`CONFIRMED`), pago dividido en curso en azul (`HELD` con el conteo `4/6`) y bloqueado por recepción en ámbar (`BLOCKED`).

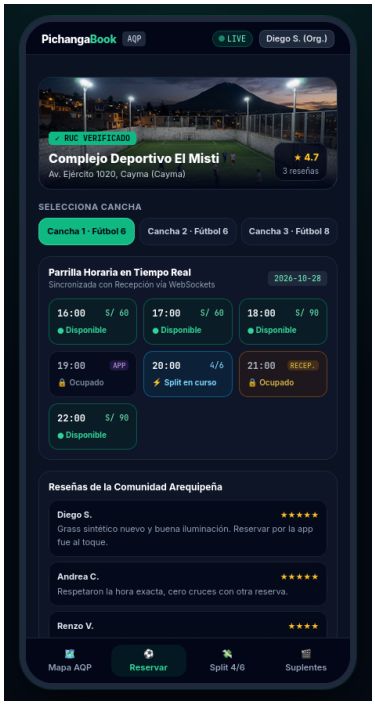


Figura 10: Parrilla horaria sincronizada y reseñas del complejo deportivo.

4.5 Pantalla de pagos divididos (Split Payment en modo demostración)



Figura 11: Contador de progreso de pago dividido (4 de 6 cupos pagados).

Esta vista consulta `GET /api/split-payments/:token` (token de prueba `pk4de6demo`), mostrando el avance en vivo (*"Cupos pagados: 4 de 6 — S/ 60.00 / S/ 90.00"*) y el botón para compartir el link por WhatsApp. Al abonar una cuota de S/ 15.00 se invoca `POST /api/split-payments/:token/pay`, que hoy simula el cargo con `CulqiService` (`chr_demo_*`), bloquea la fila en `split_payments` con `FOR UPDATE` y, al llegar a 6/6, pasa la reserva a `CONFIRMED`.

Como mejora técnica pendiente antes de producción, se corregirá `recordVerifiedShare` para que solo incremente `paid_slots` si el `provider_charge_id` no existía previamente, y se hará obligatoria la firma HMAC sobre el `rawBody` en el endpoint `/api/payments/webhooks/culqi`.

4.6 Bolsa de suplentes verificados con clips de video

La bolsa de suplentes consulta `GET /api/substitutes?position=...` en `SubstitutesController`, ejecutando un `JOIN` en PostgreSQL entre `substitute_profiles`, `users` y `districts` para filtrar jugadores disponibles por posición (`ARQUERO`, `DEFENSA`, `MEDIOCAMPO` o `DELANTERO`).

Cada tarjeta muestra la puntualidad (`punctuality_score`), partidos jugados y un reproductor con clips de 15 a 30 segundos (restringidos en BD con `CHECK (clip_duration_s BETWEEN 15 AND 30)`). Los videos mostrados provienen de los datos semilla (`seed.sql`), mientras que `POST /api/substitutes/clip-upload-signature` genera la firma SHA-1 en modo `LOCAL_DEMO` hasta configurar las llaves de Cloudinary.



Figura 12: Tarjetas de suplentes con reproductor de video (datos semilla de 15 a 30 s).

4.7 Seguridad: Validación OTP (modo demo) y chat interno con contacto protegido

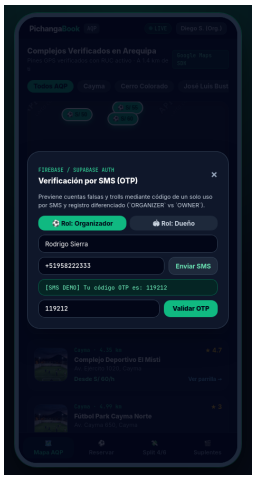


Figura 13: Registro por rol y validación de código OTP (en modo demo devuelve `devCode`).

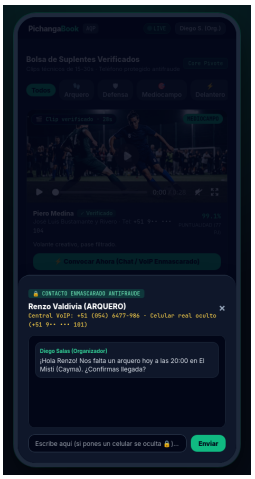


Figura 14: Chat interno con redacción de teléfonos y número proxy simulado.

En el flujo de acceso (Figura 13), `POST /api/auth/otp/request` genera un código OTP de 6 dígitos en memoria (`OtpService`) y, al estar activo `demoMode`, lo devuelve como `devCode` en pantalla sin enviar aún un SMS real; luego `POST /api/auth/otp/verify` registra o actualiza al usuario en la tabla `users` con su rol (`ORGANIZER` u `OWNER`) y emite el JWT. Por su parte, al convocar a un suplente en `POST /api/substitutes/:id/invite` (Figura 14) se crea una sesión en `contact_relays` con un `proxy_number` **simulado** (no conectado aún a una central VoIP real) y un chat interno en `relay_messages` (`POST /api/relays/:id/messages`) que oculta automáticamente cualquier teléfono o correo escrito en el mensaje (`redactContactInfo`) y muestra solo la columna generada `phone_masked` (`+51 9•• ••• 101`).

5 Avance para la Semana Siguiente (6 al 12 de noviembre de 2026)

Para la entrega de la próxima semana nos enfocaremos en reemplazar las simulaciones del modo demostración por integraciones reales y corregir los detalles de seguridad e idempotencia identificados en el backend:

Área	Tarea programada	Entregable / Evidencia para la próxima semana
Backend (Pagos y Webhook)	Integración con Culqi Sandbox real, idempotencia y firma HMAC obligatoria.	Enviar `sourceId` real desde el cliente en `createCharge`, corregir `recordVerifiedShare` para incrementar `paid_slots` solo si el cargo es nuevo, y exigir cabecera `x-culqi-signature` obligatoria sobre el `rawBody` en `/api/payments/webhooks/culqi`.
Seguridad y Autorización	Cierre de rutas `@Public()` con guards JWT/roles y SMS OTP real.	Retirar `@Public()` en las rutas sensibles de `BookingsController` y `SubstitutesController` exigiendo `JwtAuthGuard` y `@Roles()`, y conectar el envío de SMS reales con Firebase Auth (`POST /api/auth/firebase`).
Backend y BD (Reservas)	Liberación automática de reservas `HELD` vencidas (Cron Job).	Tarea programada en NestJS que cambie a `EXPIRED` las reservas `HELD` y sus `split_payments` cuando venza `hold_expires_at`, además de excluirlas en `AvailabilityService.gridForCourts`.
Multimedia y Backoffice	Subida real de clips a Cloudinary y revisión documental de complejos.	Configurar credenciales de Cloudinary en `MediaService` para subir videos reales de 15 a 30 s desde la app y desactivar la auto-aprobación demo de complejos.
Frontend Web y Móvil	Reporte de ingresos para dueños y compilación de APK en Flutter.	Añadir exportación CSV de reservas en `/web-admin-tailwind` y generar el APK instalable desde `/mobile-flutter` conectado al backend desplegado.

Tabla 1: Cronograma de tareas técnicas y entregables comprometidos para la semana del 6 al 12 de noviembre de 2026.